



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de 
HONORIS UNITED UNIVERSITIES

RAPPORT DE PROJET – MODULE DÉVELOPPEMENT MOBILE

Application Android de Quiz Géolocalisé de Rues

Firebase · FastAPI · DeepFace · GPS · Mapillary · Groq IA · Material Design 3

*Travail réalisé par :
Nafis Aymen*

Groupe : G3

Encadré par : Dr Bousmah Mohammed

Année universitaire : 2025/2026

Dédicaces

Je dédie ce travail à ma famille, dont le soutien inconditionnel et la confiance ont été ma principale source de motivation tout au long de ce projet.

À tous ceux qui, de près ou de loin, ont contribué à ma formation et à l'aboutissement de ce travail.

Remerciements

Au terme de ce travail, il m'est agréable d'exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de module Développement Mobile.

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant, Dr Bousmah Mohammed, pour sa disponibilité, ses précieux conseils et son suivi rigoureux tout au long de ce projet. Sa pédagogie et son expertise dans le domaine du développement mobile ont été une source d'inspiration indispensable.

Je remercie également l'ensemble du corps professoral de l'École Marocaine des Sciences de l'Ingénieur, dont les enseignements m'ont fourni les bases théoriques et pratiques nécessaires à l'accomplissement de ce travail.

Mes remerciements vont aussi à mes camarades de promotion pour les échanges constructifs, le partage de connaissances et le soutien moral qu'ils m'ont apportés durant toute la durée de ma formation.

Résumé

Ce rapport présente le projet de module Développement Mobile intitulé « GeoQuiz Explorer », une application mobile Android native développée en Java sous Android Studio. L'application propose une expérience de quiz interactive et originale : l'utilisateur doit identifier des rues de Casablanca à partir de photographies de niveau rue issues de l'API Mapillary.

L'application s'appuie sur Firebase comme plateforme de backend cloud, en utilisant Firebase Authentication pour la gestion sécurisée des comptes utilisateurs, enrichie d'une authentification biométrique faciale via un backend personnel FastAPI et le modèle DeepFace Facenet. Cloud Firestore assure la persistance des données et le stockage des questions.

Cinq rues de Casablanca constituent la base de données de questions : Avenue Hassan II, Rue Socrates, Boulevard d'Afghanistan, Boulevard Sidi Mohammed Ben Abdellah, et Avenue des Forces Armées Royales (FAR). L'application intègre également un assistant conversationnel alimenté par l'IA Groq (modèle LLaMA 3.3 70B), avec saisie vocale via le SpeechRecognizer Android. L'interface utilisateur est conçue selon les principes du Material Design 3.

Mots-clés : *Android, Java, Firebase, FastAPI, DeepFace, Reconnaissance Faciale, Firestore, GPS, Mapillary, Groq, LLaMA, IA Conversationnelle, Saisie Vocale, Material Design 3, Quiz de Rues, Casablanca.*

Abstract

This report presents the Mobile Development module project entitled 'GeoQuiz Explorer', a native Android mobile application developed in Java using Android Studio. The application offers an interactive and original quiz experience: the user must identify streets of Casablanca from street-level photographs sourced from the Mapillary API.

The application relies on Firebase as a cloud backend platform, using Firebase Authentication for secure user account management, enhanced with facial biometric authentication via a personal FastAPI backend and the DeepFace Facenet model. The application also integrates a conversational assistant powered by Groq AI (LLaMA 3.3 70B model), with voice input via Android SpeechRecognizer.

Keywords: *Android, Java, Firebase, FastAPI, DeepFace, Face Recognition, Firestore, GPS, Mapillary, Groq, LLaMA, Conversational AI, Voice Input, Material Design 3, Street Quiz, Casablanca.*

Liste des Abréviations

Abréviation	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
APK	Android Package Kit
ASGI	Asynchronous Server Gateway Interface
BOM	Bill of Materials
CameraX	Bibliothèque Android pour la gestion de la caméra
CORS	Cross-Origin Resource Sharing
FastAPI	Framework Python pour la création d'APIs REST
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MD3	Material Design 3
REST	Representational State Transfer
SDK	Software Development Kit
UI	User Interface
URL	Uniform Resource Locator

Liste des Figures

Figure 1 : Architecture générale de l'application GeoQuiz Explorer	1
Figure 2 : Diagramme de Gantt – Planification du projet.....	3
Figure 3 : Diagramme de cas d'utilisation – GeoQuiz Explorer	5
Figure 4 : Diagramme d'activité – Flux d'authentification biométrique	6
Figure 5 : Diagramme de classes simplifié – GeoQuiz Explorer	6
Figure 6 : Diagramme de navigation entre les activités Android	8
Figure 7 : Architecture Firebase (Authentication + Firestore).....	9
Figure 8 : Architecture du backend FastAPI – Reconnaissance faciale	9
Figure 9 : Logo Android Studio.....	11
Figure 10 : Logo Java.....	11
Figure 11 : Logo Firebase	12
Figure 12 : Logo Google Play Services	12
Figure 13 : Logo Mapillary	12
Figure 14 : Logo Groq Cloud – LLaMA 3.3 70B	13
Figure 15 : Logo FastAPI	13
Figure 16 : Logo DeepFace.....	13
Figure 17 : Capture d'écran : Écran de connexion (MainActivity).....	15
Figure 18 : Capture d'écran : Écran de capture faciale (FaceCaptureActivity)	16
Figure 19 : Capture d'écran : Écran d'inscription (RegisterActivity).....	17
Figure 20 : Console Cloud Firestore (structure quizzes/casablanca/questions).....	18
Figure 21 : Capture de l'écran de demande d'activation de localisation GPS	18
Figure 22 : Flux de détection GPS et géolocalisation.....	19
Figure 23 : Exemple de photographie Mapillary	20
Figure 24 : Capture d'écran : Interface de quiz (QuizActivity)	21
Figure 25 : Capture d'écran : Assistant IA avec saisie vocale (ChatActivity).....	22
Figure 26 : Capture d'écran : Écran des résultats (ScoreActivity)	23

Liste des Tableaux

Tableau 1 : Périmètre IN / OUT du projet.....	2
Tableau 2 : Fichiers et leurs responsabilités	10
Tableau 3 : Technologies utilisées et leurs rôles.....	10
Tableau 4 : Endpoints FastAPI du backend facial	16
Tableau 5 : Structure Firestore : collection quizzes/casablanca/questions	19
Tableau 6 : Les 5 rues de Casablanca avec pKey Mapillary	20
Tableau 7 : Structure Firestore : collection users	20

Table des Matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iii
Liste des Abréviations	iv
Liste des Figures	v
Liste des Tableaux	vi
Table des Matières	vii
Introduction Générale	1
Structure du Rapport	1
Chapitre 1 : Contexte Général du Projet	2
I. Présentation du Projet	2
I.1 Problématique	2
I.2 Solution Proposée	2
II. Méthodologie de Travail et Planification	3
Chapitre 2 : Analyse et Conception	4
Introduction	4
I. Étude Fonctionnelle	4
I.1 Analyse des Besoins Fonctionnels	4
I.2 Analyse des Besoins Non Fonctionnels	4
II. Étude Conceptuelle	4
II.1 Règles de Gestion	4
II.2 Diagrammes UML	5
Conclusion	7
Chapitre 3 : Étude Technique	8
Introduction	8
I. Architecture du Projet	8
II. Technologies Utilisées	10
III. Environnement de Travail	13
Conclusion	14
Chapitre 4 : Mise en Œuvre	15
Introduction	15
I. Réalisation	15
I.1 Authentification Biométrique Faciale	15
I.2 Inscription et Enregistrement Facial	16

I.3 Géolocalisation GPS	17
I.4 Moteur de Quiz et Intégration Mapillary	19
I.5 Assistant IA Conversationnel avec Saisie Vocale	21
I.6 Résultats et Classement	22
II. DevOps (Déploiement et Infrastructure)	23
Conclusion	24
Conclusion Générale et Perspectives	25
Perspectives et Améliorations Futures	25
Bibliographie et Références	26
Documentation Officielle	26
Ouvrages de Référence.....	26
Ressources en Ligne	26

Introduction Générale

Le développement mobile constitue aujourd'hui l'un des domaines les plus dynamiques de l'ingénierie informatique. Avec plus de trois milliards de smartphones actifs dans le monde, les applications mobiles sont devenues des outils incontournables du quotidien, transformant profondément nos modes d'apprentissage, de divertissement et d'interaction sociale.

C'est dans ce contexte en perpétuelle évolution que s'inscrit le présent projet de module Développement Mobile : « GeoQuiz Explorer ». Cette application Android, développée en Java natif et intégrée à l'écosystème Google (Firebase, Google Play Services), propose une approche innovante du quiz culturel en exploitant les photographies de niveau rue de l'API Mapillary pour mettre l'utilisateur au défi d'identifier des rues de Casablanca.

Au-delà du quiz, l'application intègre plusieurs couches technologiques avancées : une authentification biométrique faciale via un backend FastAPI personnel utilisant le modèle DeepFace Facenet, un assistant conversationnel alimenté par l'IA (LLaMA 3.3 70B via Groq), et une saisie vocale dans le chat via le SpeechRecognizer Android.

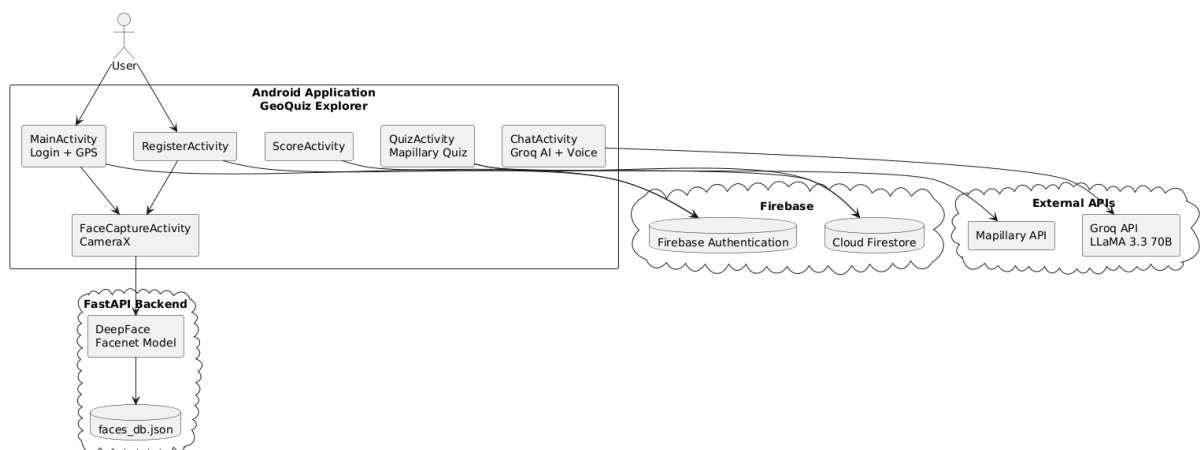


Figure 1 : Architecture générale de l'application GeoQuiz Explorer

Structure du Rapport

Le présent rapport est organisé en quatre chapitres complémentaires :

- Le Chapitre 1 présente le contexte général du projet, la problématique, la solution proposée, et la méthodologie avec le diagramme de Gantt.
- Le Chapitre 2 présente l'analyse et la conception : besoins fonctionnels, règles de gestion, et diagrammes UML.
- Le Chapitre 3 expose l'étude technique : architecture, technologies utilisées, et environnement de travail.
- Le Chapitre 4 décrit la mise en œuvre complète : réalisation de l'application et déploiement DevOps.

Chapitre 1 : Contexte Général du Projet

Ce chapitre présente le contexte dans lequel s'inscrit le projet GeoQuiz Explorer, la problématique identifiée, la solution proposée, ainsi que la méthodologie de travail adoptée.

I. Présentation du Projet

I.1 Problématique

Les applications de quiz disponibles sur le marché proposent généralement des questions textuelles génériques, sans ancrage visuel dans la réalité urbaine. Cette approche présente plusieurs limitations :

- Absence d'immersion visuelle dans l'environnement géographique réel.
- Contenu statique ne reflétant pas l'apparence réelle des lieux.
- Manque d'engagement et de défi cognitif vis-à-vis de la connaissance urbaine locale.
- Absence de mécanismes d'authentification biométrique sécurisée dans les applications éducatives mobiles.

La problématique centrale peut être formulée ainsi : comment concevoir une application de quiz mobile utilisant des photographies réelles de rues, sécurisée par reconnaissance faciale, enrichie d'un assistant IA conversationnel avec saisie vocale, tout en intégrant un backend personnel ?

I.2 Solution Proposée

GeoQuiz Explorer répond à cette problématique en combinant plusieurs technologies complémentaires :

- Un quiz visuel basé sur des photographies de rue réelles issues de l'API Mapillary.
- Une authentification biométrique faciale via un backend FastAPI personnel et le modèle DeepFace Facenet.
- Un assistant IA conversationnel (LLaMA 3.3 70B via Groq) avec saisie vocale.
- Une géolocalisation GPS pour adapter le contenu à la ville de l'utilisateur.
- Un backend cloud Firebase pour l'authentification et la persistance des données.

Tableau 1 : Périmètre IN / OUT du projet

Périmètre IN	Périmètre OUT
Authentification email/mot de passe + biométrie faciale	Classement mondial en temps réel
Détection GPS (Casablanca)	Partage social des scores
Quiz 5 questions avec photos Mapillary interactives	Mode hors ligne complet
Assistant IA Groq avec saisie vocale	Support multi-langues

Périmètre IN	Périmètre OUT
Backend FastAPI avec reconnaissance faciale DeepFace	Notifications push
Interface Material Design 3	Extension à d'autres villes (prévu en perspectives)

II. Méthodologie de Travail et Planification

Le projet a été développé selon une approche itérative et incrémentale, permettant d'intégrer progressivement les fonctionnalités tout en maintenant une application fonctionnelle à chaque étape. La méthodologie adoptée s'appuie sur les principes du développement Agile, avec des cycles courts de développement-test-intégration.

Les outils utilisés pour la gestion du projet incluent Android Studio avec GitHub Copilot pour l'assistance au développement, Git et GitHub pour le contrôle de version, et Firebase Console pour la gestion du backend cloud.

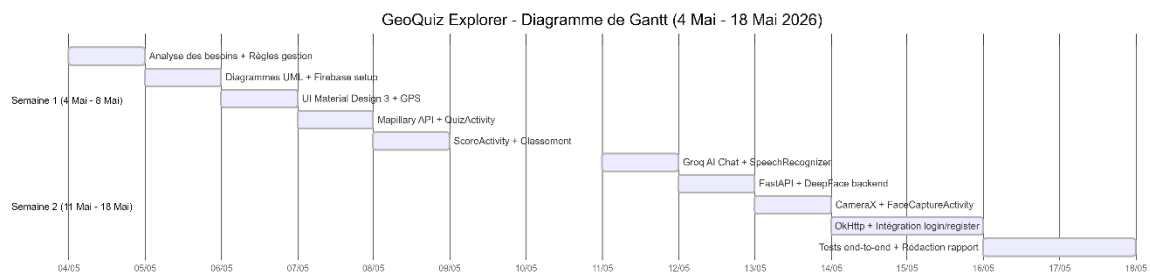


Figure 2 : Diagramme de Gantt – Planification du projet

Chapitre 2 : Analyse et Conception

Ce chapitre présente l'analyse fonctionnelle et conceptuelle du projet GeoQuiz Explorer, depuis l'identification des besoins jusqu'à la modélisation UML du système.

Introduction

L'analyse et la conception constituent une étape fondamentale du cycle de développement logiciel. Elles permettent de formaliser les besoins, de définir les règles de gestion, et de produire des modèles précis avant l'implémentation.

I. Étude Fonctionnelle

I.1 Analyse des Besoins Fonctionnels

Les besoins fonctionnels de GeoQuiz Explorer ont été identifiés en analysant les objectifs du projet et les attentes des utilisateurs cibles :

- Authentification sécurisée : inscription et connexion par email/mot de passe, renforcée par une vérification biométrique faciale à chaque connexion et enregistrement.
- Quiz géolocalisé : présentation de 5 questions basées sur des photographies réelles de rues de Casablanca, avec 4 choix de réponse par question.
- Interaction visuelle : affichage interactif 360° des photographies de rue via WebView Mapillary.
- Résultats et classement : affichage du score final, mise à jour du meilleur score personnel, et classement des 5 meilleurs joueurs.
- Assistant IA conversationnel : possibilité de poser des questions textuelles ou vocales sur les rues du quiz à un assistant basé sur LLaMA 3.3 70B.
- Géolocalisation : détection automatique de la ville de l'utilisateur pour adapter le quiz.

I.2 Analyse des Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles doit répondre l'application :

- Performance : temps de réponse de l'IA et de la reconnaissance faciale acceptables (< 10 secondes après le premier chargement du modèle).
- Sécurité : les clés API sont stockées dans secrets.properties (hors contrôle de version), et l'API Firebase est restreinte au package de l'application.
- Maintenabilité : architecture modulaire avec séparation des responsabilités entre activités.
- Compatibilité : application ciblant Android API 24+ (Android 7.0 et supérieur).
- Accessibilité : interface respectant les standards Material Design 3 pour une expérience utilisateur optimale.

II. Étude Conceptuelle

II.1 Règles de Gestion

Les règles de gestion définissent les contraintes métier régissant le comportement de l'application :

- RG01 – Un utilisateur ne peut se connecter qu'avec un email et un mot de passe valides ET une correspondance faciale positive (distance Euclidienne Facenet < 10).
- RG02 – Lors de l'inscription, le visage de l'utilisateur est obligatoirement capturé et stocké sous forme d'embedding vectoriel dans le backend FastAPI.
- RG03 – En cas d'échec de la reconnaissance faciale à la connexion, l'accès est refusé indépendamment des credentials Firebase.
- RG04 – Le quiz comprend exactement 5 questions, chargées depuis Firestore, présentées dans un ordre fixe.
- RG05 – Les 4 options de réponse sont mélangées aléatoirement à chaque affichage pour éviter les biais positionnels.
- RG06 – Le meilleur score (highScore) est mis à jour uniquement si le nouveau score est strictement supérieur au score précédent.
- RG07 – Le classement affiche les 5 utilisateurs ayant le meilleur highScore, par ordre décroissant.
- RG08 – Si la permission GPS est refusée ou si le timeout de 15 secondes est atteint, l'application bascule automatiquement sur la ville par défaut (Casablanca).
- RG09 – La saisie vocale remplit uniquement le champ de texte ; l'envoi reste manuel pour permettre à l'utilisateur de vérifier.

II.2 Diagrammes UML

Les diagrammes UML suivants modélisent le comportement et la structure du système GeoQuiz Explorer.

Diagramme de Cas d'Utilisation

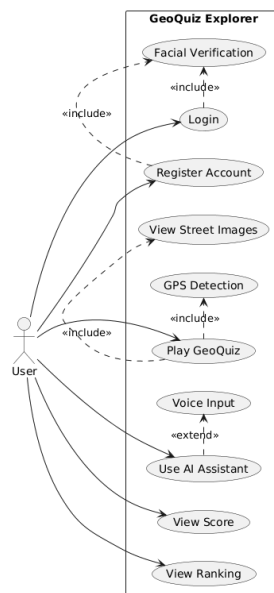


Figure 3 : Diagramme de cas d'utilisation – GeoQuiz Explorer

Diagramme d'Activité

Le diagramme d'activité ci-dessous illustre le flux complet d'authentification biométrique, de la saisie des credentials jusqu'à l'accès au quiz.

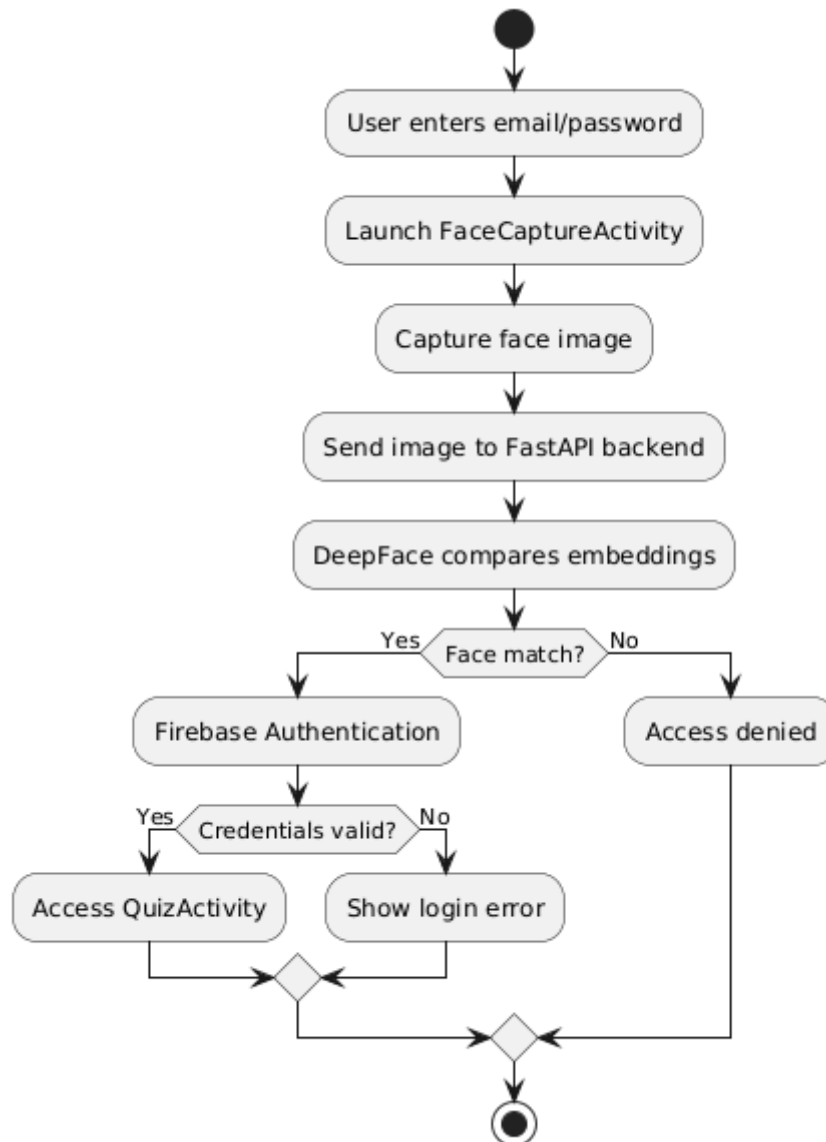


Figure 4 : Diagramme d'activité – Flux d'authentification biométrique

Diagramme de Classes

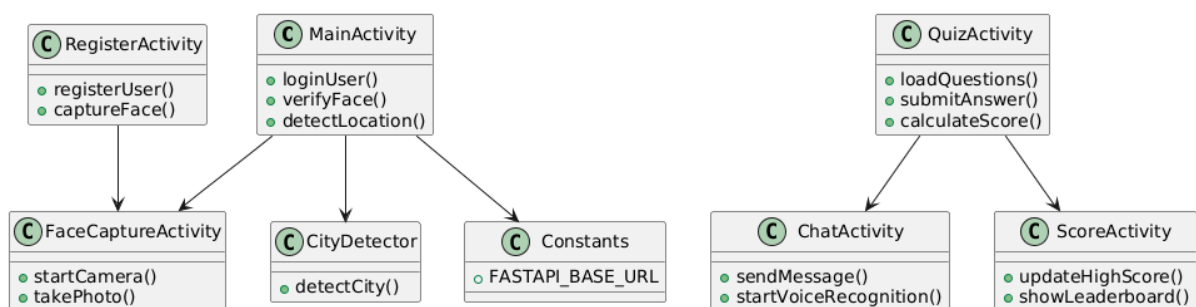


Figure 5 : Diagramme de classes simplifié – GeoQuiz Explorer

Conclusion

Cette analyse a permis de formaliser les besoins fonctionnels et non fonctionnels de l'application, de définir les règles de gestion, et de produire les modèles UML. Le chapitre suivant présente l'étude technique qui découle de cette conception.

Chapitre 3 : Étude Technique

Ce chapitre présente l'architecture technique de GeoQuiz Explorer, les technologies utilisées, et l'environnement de travail mis en place pour le développement.

Introduction

Le choix des technologies conditionne directement la faisabilité, la performance et la maintenabilité d'une application. Ce chapitre justifie les choix techniques effectués et décrit l'architecture globale du système.

I. Architecture du Projet

L'architecture de GeoQuiz Explorer repose sur un modèle client-cloud enrichi d'un backend personnel et d'APIs externes. On distingue quatre couches logiques :

- Couche Présentation (UI) : Ensemble des activités Android (MainActivity, RegisterActivity, FaceCaptureActivity, QuizActivity, ScoreActivity, ChatActivity) et de leurs layouts XML Material Design 3.
- Couche Logique Métier : Logique de quiz, gestion du score, détection de ville, reconnaissance faciale, appels aux APIs Mapillary, Groq et FastAPI.
- Couche Backend Personnel : Serveur FastAPI local exposé via Cloudflare Tunnel, hébergeant les endpoints de reconnaissance faciale DeepFace.
- Couche Données : Cloud Firestore + faces_db.json (embeddings faciaux) + APIs externes (Mapillary, Groq).

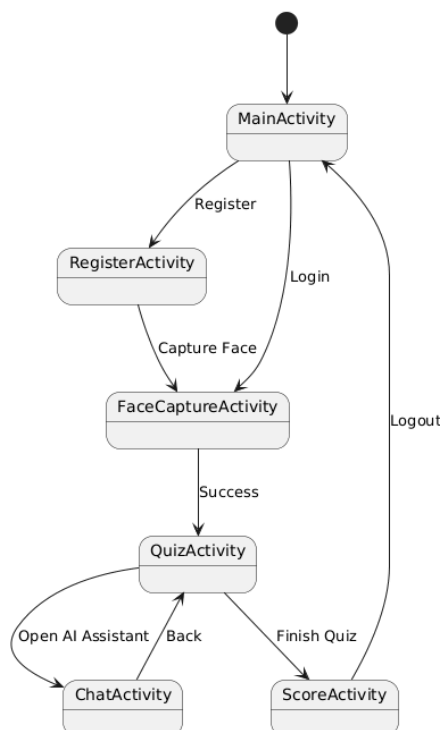


Figure 6 : Diagramme de navigation entre les activités Android

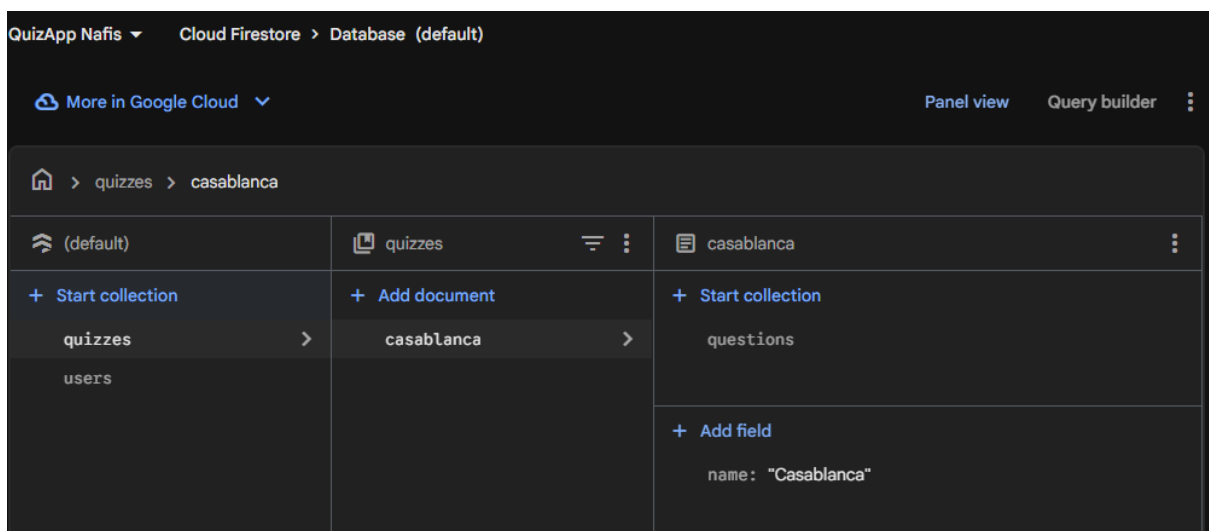
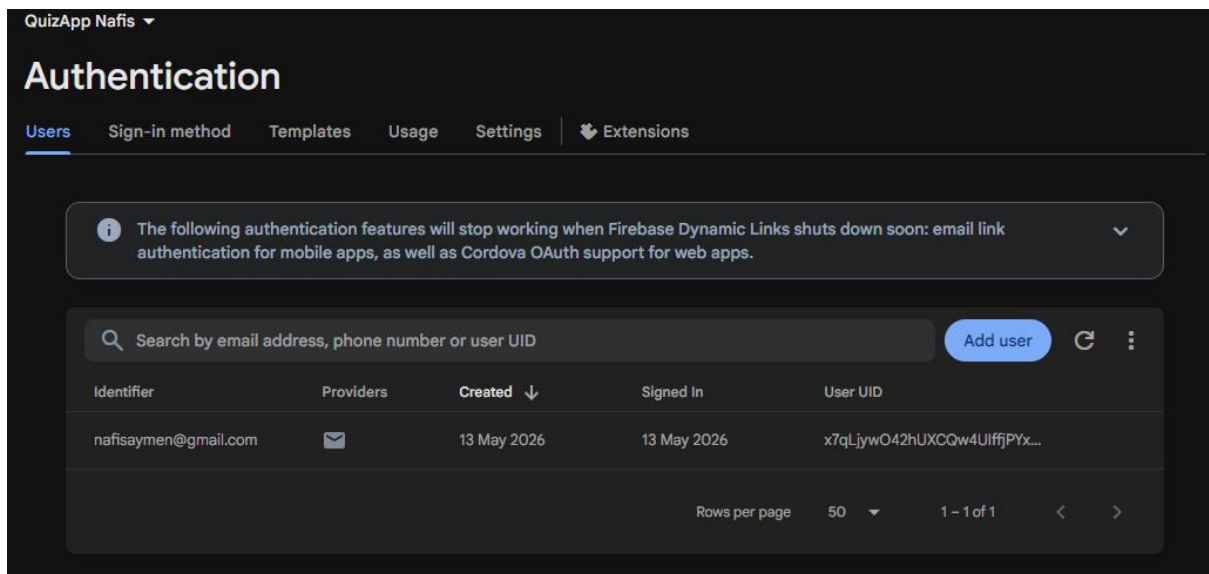


Figure 7 : Architecture Firebase (Authentication + Firestore)

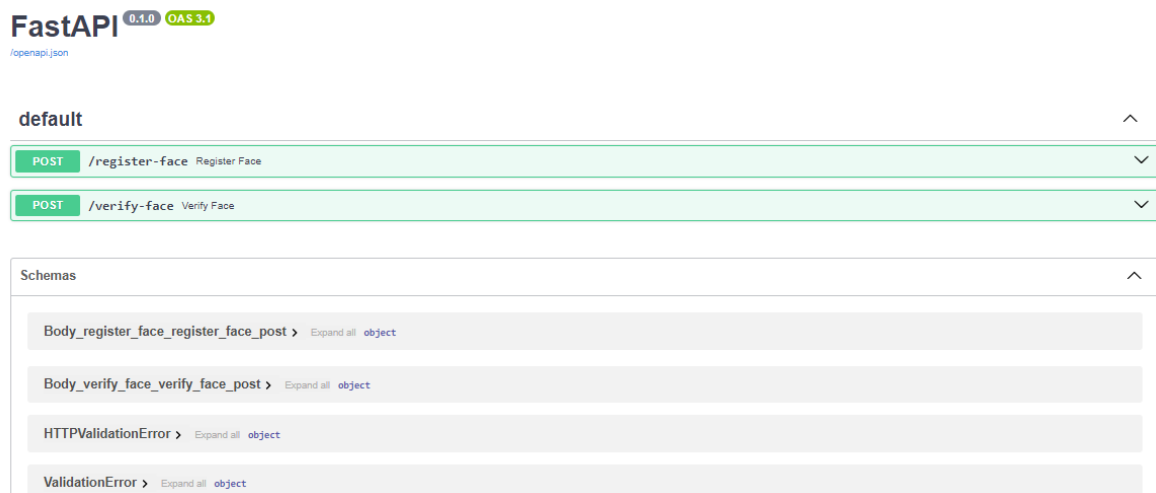


Figure 8 : Architecture du backend FastAPI – Reconnaissance faciale

Tableau 2 : Fichiers et leurs responsabilités

Dossier / Fichier	Contenu / Responsabilité
java/.../MainActivity.java	Écran de connexion, flux face scan, authentification Firebase, GPS
java/.../RegisterActivity.java	Inscription Firebase, capture et enregistrement du visage
java/.../FaceCaptureActivity.java	Capture photo via CameraX, retour du chemin d'image
java/.../QuizActivity.java	Moteur de quiz, chargement Firestore, WebView Mapillary
java/.../ScoreActivity.java	Affichage résultats, mise à jour highScore, classement top 5
java/.../ChatActivity.java	Assistant IA Groq, saisie vocale SpeechRecognizer
java/.../CityDetector.java	Calcul distance GPS, détection ville par défaut
java/.../Constants.java	FASTAPI_BASE_URL centralisé
res/layout/	Tous les fichiers XML de mise en page
res/values/	themes.xml (MD3), strings.xml, colors.xml
AndroidManifest.xml	Activités, permissions INTERNET, LOCATION, CAMERA, RECORD_AUDIO
geoquiz_backend/main.py	FastAPI : /register-face et /verify-face avec DeepFace
secrets.properties	Clé API Groq (hors contrôle de version)
libs.versions.toml	Version Catalog : Firebase BOM, OkHttp, CameraX, Play Services

II. Technologies Utilisées

Tableau 3 : Technologies utilisées et leurs rôles

Technologie / Outil	Rôle dans le Projet	Version
Java (Android Native)	Logique métier, activités, traitement des données	Java 11+
Android Studio	IDE, compilation, débogage, publication	Hedgehog+
GitHub Copilot	Assistance à la génération de code	2025
Firebase Authentication	Inscription, connexion, gestion des sessions	BOM 32+
Cloud Firestore	Questions, profils utilisateurs, scores	BOM 32+

Technologie / Outil	Rôle dans le Projet	Version
FastAPI (Python)	Backend personnel – endpoints reconnaissance faciale	0.100+
DeepFace (Facenet)	Génération et comparaison d'embeddings faciaux	0.0.100
CameraX	Aperçu caméra temps réel et capture photo	1.3.0
FusedLocationProviderClient	Géolocalisation haute précision	GMS 21+
Mapillary API	Photos de niveau rue pour les questions de quiz	Graph API v4
OkHttp	Requêtes HTTP (Mapillary, Groq, FastAPI)	4.12.0
Groq API (LLaMA 3.3 70B)	Assistant IA conversationnel	2025
SpeechRecognizer Android	Saisie vocale dans ChatActivity	API 23+
Cloudflare Tunnel	Exposition du backend local via HTTPS public	2026.5.0
Material Design 3	Composants UI, thèmes, animations	MDC-Android 1.9+
Gradle + Version Catalog	Gestion des dépendances et build	Gradle 8+



Figure 9 : Logo Android Studio



Figure 10 : Logo Java



Figure 11 : Logo Firebase

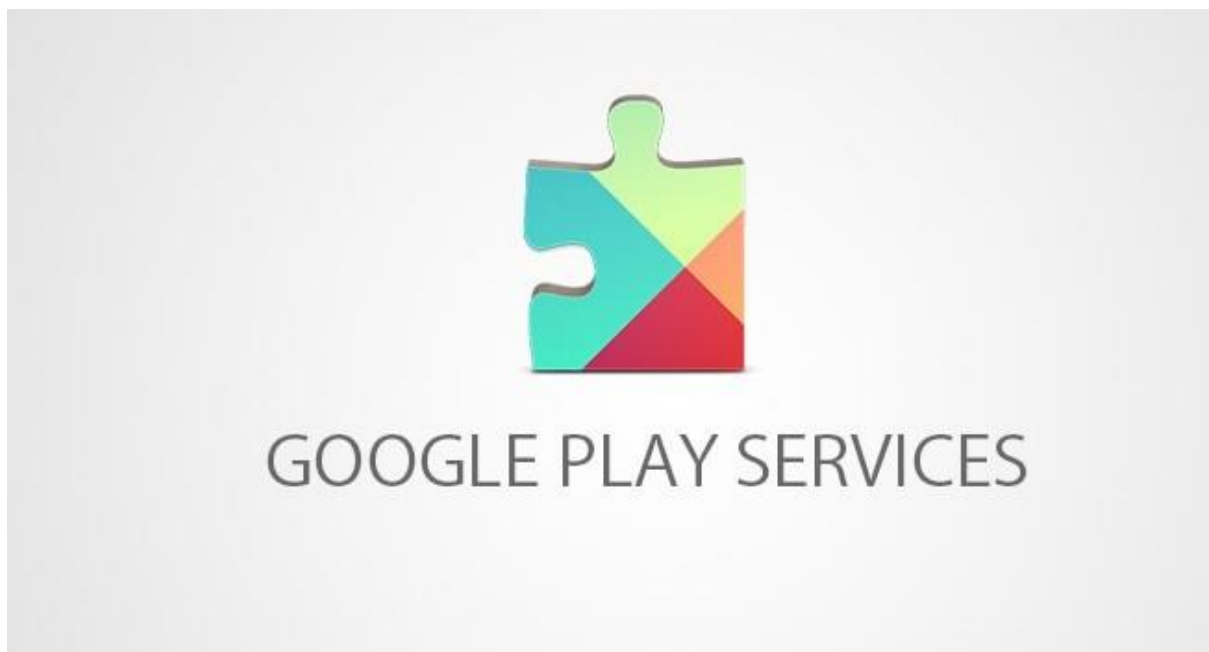


Figure 12 : Logo Google Play Services



Figure 13 : Logo Mapillary



Figure 14 : Logo Groq Cloud – LLaMA 3.3 70B



Figure 15 : Logo FastAPI



Figure 16 : Logo DeepFace

III. Environnement de Travail

Le développement de GeoQuiz Explorer a nécessité la mise en place d'un environnement de travail structuré, combinant des outils locaux et des services cloud.

- Poste de développement : Windows 11, Android Studio Hedgehog, Python 3.13, Git.
- Appareil de test : téléphone Android physique connecté en USB avec débogage USB activé.
- Contrôle de version : dépôt GitHub (<https://github.com/Aymen21-n/QuizApp-GeoQuiz-Explorer-Nafis>).
- Backend local : FastAPI sur port 8000, exposé via Cloudflare Tunnel sur URL HTTPS publique.
- Services cloud : Firebase Console, Groq Console, Mapillary (token d'accès).

Conclusion

L'étude technique a permis de définir une architecture solide et modulaire pour GeoQuiz Explorer. Les technologies choisies couvrent l'ensemble des besoins fonctionnels identifiés. Le chapitre suivant présente la mise en œuvre concrète de cette architecture.

Chapitre 4 : Mise en Œuvre

Ce chapitre présente la réalisation concrète de l'application GeoQuiz Explorer, en décrivant l'implémentation de chaque fonctionnalité et les captures d'écran illustrant le résultat final.

Introduction

La mise en œuvre traduit les choix de conception et les décisions techniques en code fonctionnel. Ce chapitre suit le flux de l'application, de l'authentification biométrique jusqu'à l'assistant IA conversationnel.

I. Réalisation

I.1 Authentification Biométrique Faciale

L'authentification constitue le point d'entrée de l'application. Elle combine Firebase Authentication (email/mot de passe) avec une vérification biométrique faciale via le backend FastAPI personnel.

Le flux de connexion se déroule en trois étapes : (1) l'utilisateur saisit ses credentials, (2) FaceCaptureActivity ouvre la caméra CameraX pour capturer le visage, (3) l'image est envoyée via OkHttp au endpoint /verify-face du backend FastAPI qui utilise DeepFace Facenet pour calculer la distance Euclidienne entre l'embedding capturé et l'embedding stocké. Si la distance est inférieure à 10, la connexion Firebase est autorisée.

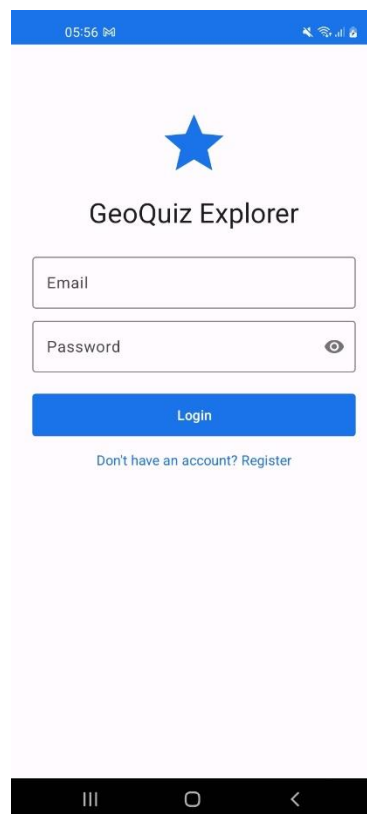


Figure 17 : Capture d'écran : Écran de connexion (MainActivity)



Figure 18 : Capture d'écran : Écran de capture faciale (FaceCaptureActivity)

Tableau 4 : Endpoints FastAPI du backend facial

Endpoint	Méthode	Description	Réponse
/register-face	POST	Génère et stocke l'embedding facial pour un nouvel utilisateur	{"status": "registered"}
/verify-face	POST	Compare le visage capturé avec l'embedding stocké	{"match": true/false, "distance": float}

I.2 Inscription et Enregistrement Facial

L'inscription crée un compte Firebase puis déclenche immédiatement la capture faciale. L'embedding généré par DeepFace est stocké dans faces_db.json côté serveur, indexé par l'email de l'utilisateur. En cas d'échec de détection faciale, le compte Firebase est supprimé pour maintenir la cohérence des données.

05:56

Create Account

Full Name

Email

Password

Confirm Password

Register

[Already have an account? Login](#)

Figure 19 : Capture d'écran : Écran d'inscription (RegisterActivity)

I.3 Géolocalisation GPS

Après connexion réussie, l'application demande la permission `ACCESS_FINE_LOCATION`. `FusedLocationProviderClient` obtient la dernière position connue via `getLastLocation()`. Si le résultat est null, `requestLocationUpdates()` déclenche une mise à jour active avec un timeout de 15 secondes. En cas de dépassement ou de permission refusée, `CityDetector.getDefaultCity()` retourne « casablanca » comme ville par défaut.

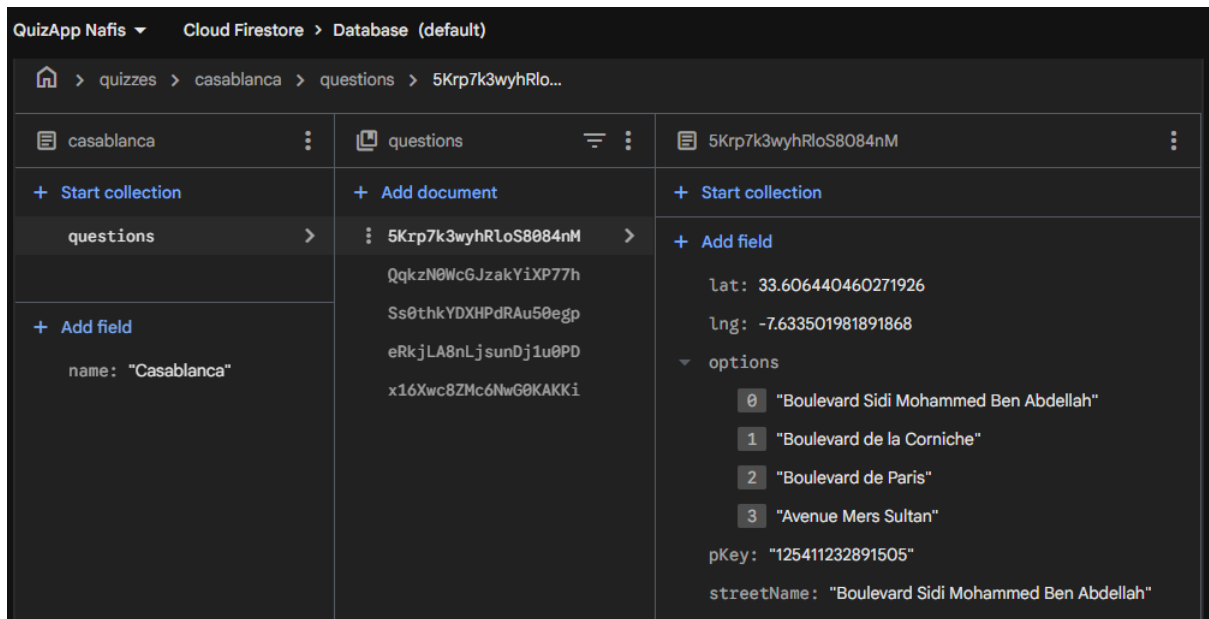


Figure 20 : Console Cloud Firestore (structure quizzes/casablanca/questions)

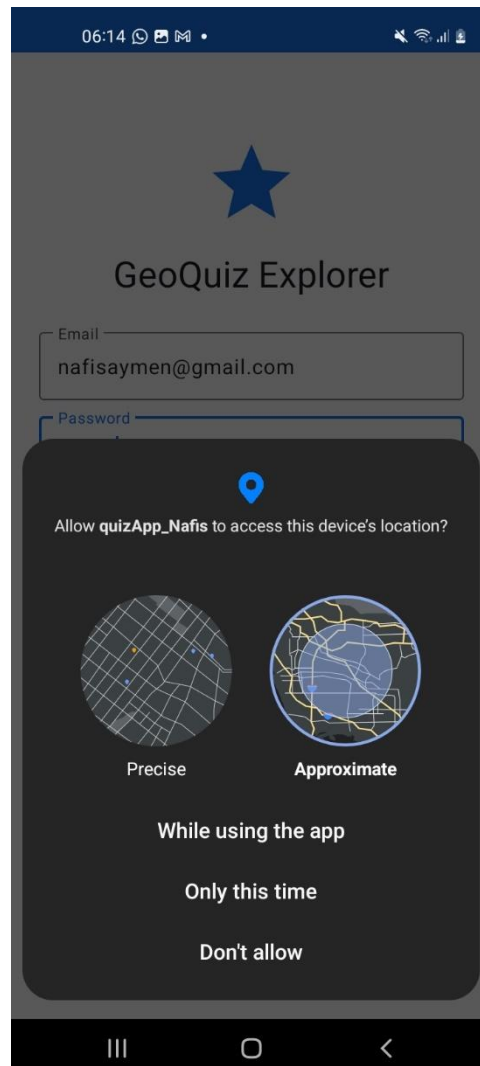


Figure 21 : Capture de l'écran de demande d'activation de localisation GPS

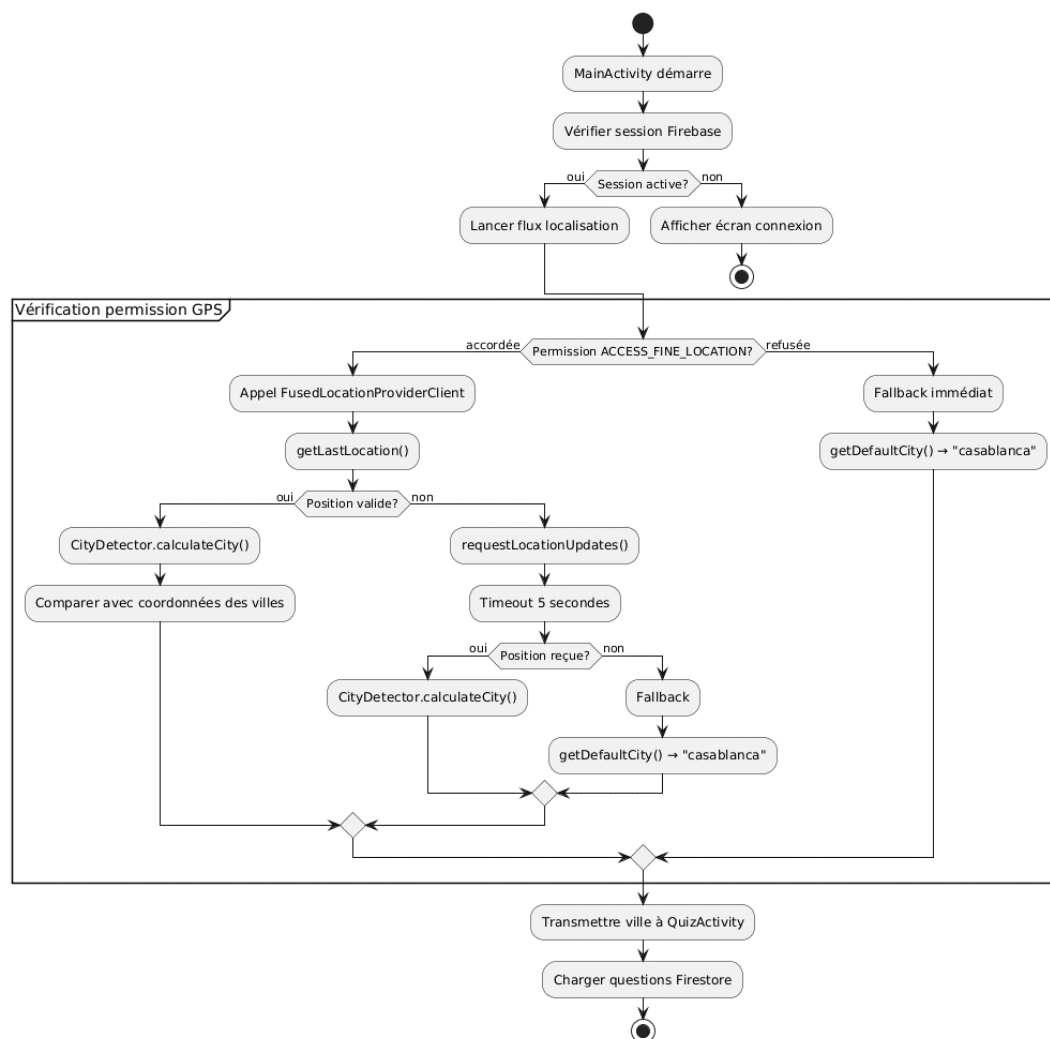


Figure 22 : Flux de détection GPS et géolocalisation

I.4 Moteur de Quiz et Intégration Mapillary

QuizActivity charge les 5 questions depuis la collection Firestore quizzes/{city}/questions. Pour chaque question, showQuestion() mélange les options via Collections.shuffle() et charge la photographie Mapillary dans un WebView interactif :

```
https://www.mapillary.com/embed?image_key={pKey}&style=photo
```

Tableau 5 : Structure Firestore : collection quizzes/casablanca/questions

Champ	Type	Description
streetName	String	Nom correct de la rue (réponse attendue)
lat	Number	Latitude de l'image Mapillary
lng	Number	Longitude de l'image Mapillary
options	Array	4 choix de réponse (correct en index 0, mélangé en code)

Champ	Type	Description
pKey	String	Identifiant image Mapillary pour le chargement WebView

Tableau 6 : Les 5 rues de Casablanca avec pKey Mapillary

#	Rue	pKey Mapillary
1	Avenue Hassan II	770637766976596
2	Rue Socrates	819480988680673
3	Boulevard d'Afghanistan	2833593923619131
4	Boulevard Sidi Mohammed Ben Abdellah	125411232891505
5	Avenue des Forces Armées Royales (FAR)	373963160666195

Tableau 7 : Structure Firestore : collection users

Champ	Type	Description
name	String	Nom complet de l'utilisateur
email	String	Adresse email de l'utilisateur
highScore	Number	Meilleur score obtenu (sur 5)

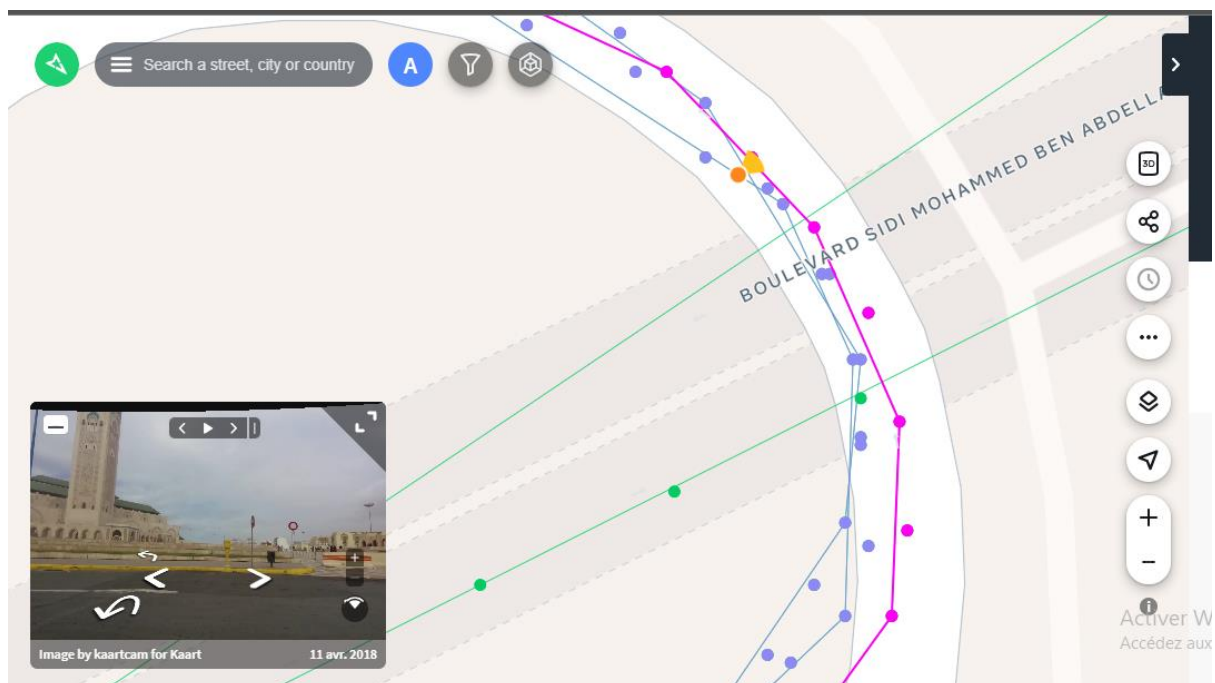


Figure 23 : Exemple de photographie Mapillary

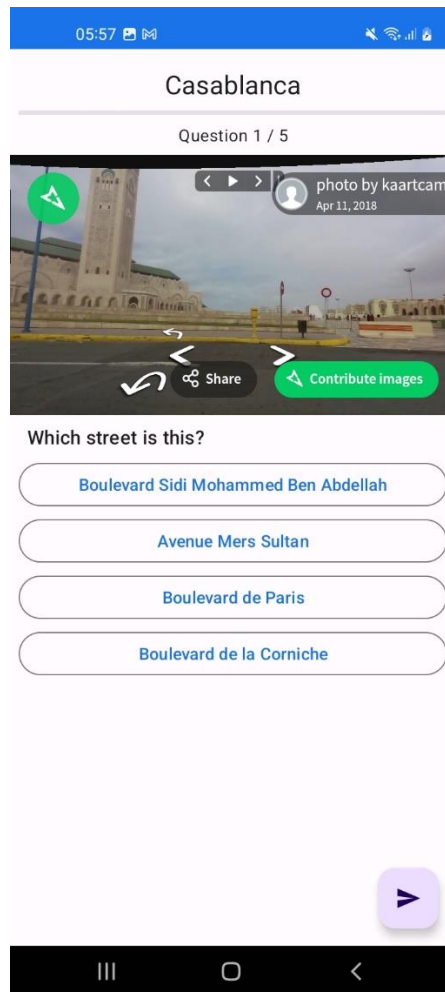


Figure 24 : Capture d'écran : Interface de quiz (QuizActivity)

I.5 Assistant IA Conversationnel avec Saisie Vocale

ChatActivity implémente un assistant conversationnel multi-tours basé sur l'API Groq (LLaMA 3.3 70B). L'historique complet de la conversation est transmis à chaque requête pour maintenir le contexte. La saisie vocale est intégrée via le SpeechRecognizer Android : après validation de la permission RECORD_AUDIO, le dialogue de reconnaissance vocale natif s'ouvre, et le texte reconnu remplit le champ de saisie sans envoi automatique.

Permission	Utilisation
INTERNET	Appels APIs Firebase, FastAPI, Mapillary, Groq
ACCESS_FINE_LOCATION	Détection GPS de la ville de l'utilisateur
CAMERA	Capture photo pour la reconnaissance faciale
RECORD_AUDIO	Saisie vocale dans ChatActivity

Tableau 8 – Récapitulatif des permissions Android requises

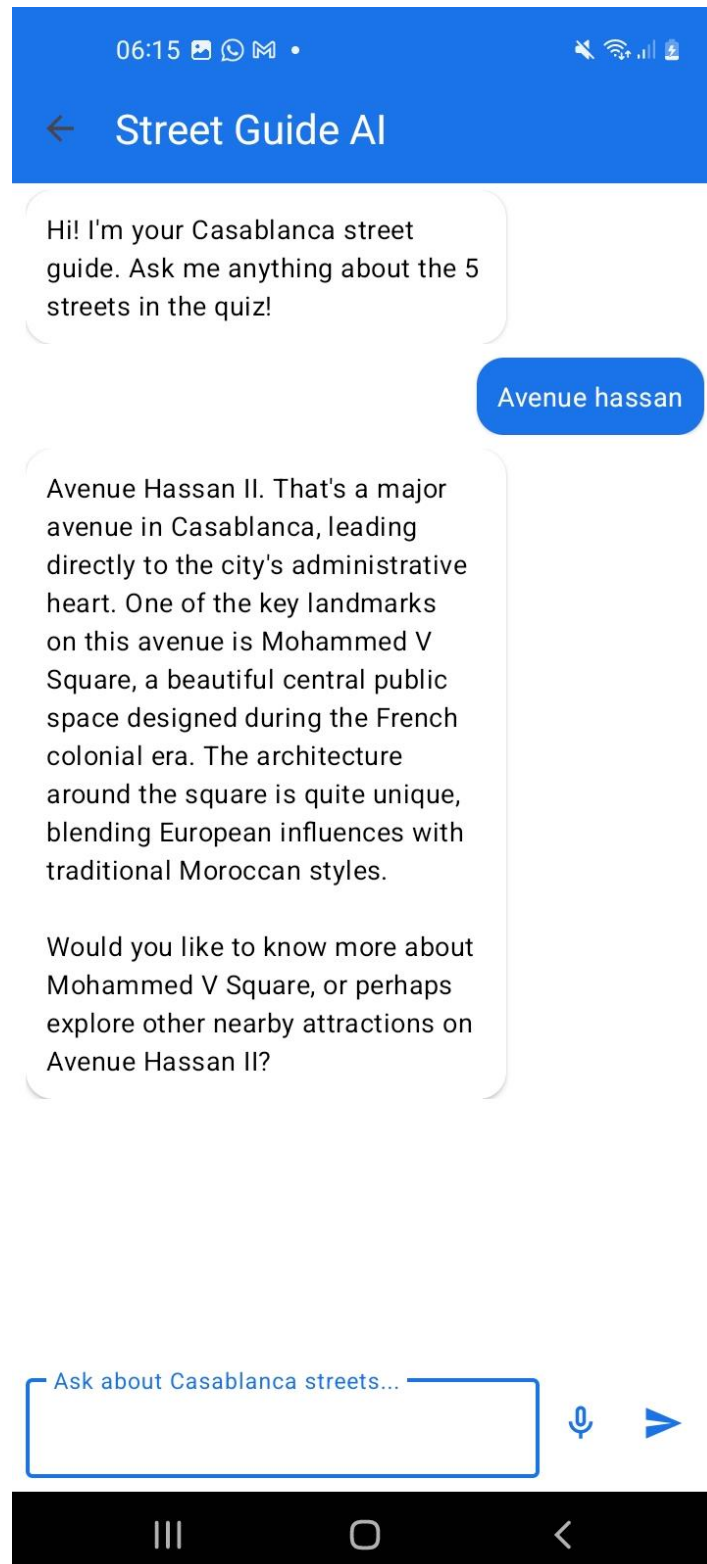


Figure 25 : Capture d'écran : Assistant IA avec saisie vocale (ChatActivity)

I.6 Résultats et Classement

ScoreActivity affiche le score final, met à jour le meilleur score dans Firestore si nécessaire, et présente un classement top 5 via RecyclerView. L'animation de la barre de progression utilise ObjectAnimator pour une présentation dynamique des résultats.

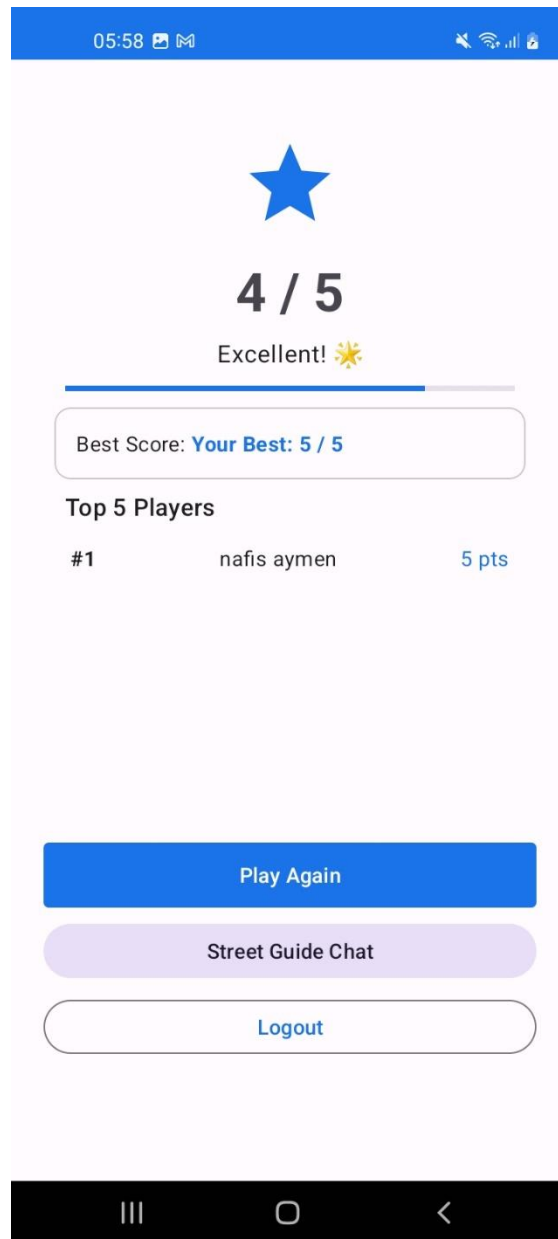


Figure 26 : Capture d'écran : Écran des résultats (ScoreActivity)

II. DevOps (Déploiement et Infrastructure)

Le déploiement de GeoQuiz Explorer implique la gestion simultanée de plusieurs composants infrastructurels :

- Backend FastAPI : serveur local démarré via uvicorn sur le port 8000, exposé publiquement via Cloudflare Tunnel (URL HTTPS dynamique mise à jour dans Constants.java).
- Firebase : configuration via google-services.json (exclu du contrôle de version), clé API restreinte au package com.example.quizapp_nafis et à la liste des APIs Firebase.
- Gestion des secrets : clé API Groq stockée dans secrets.properties (ignoré par .gitignore), template secrets.properties.example fourni dans le dépôt.
- Contrôle de version : dépôt GitHub avec .gitignore configuré pour exclure google-services.json, faces_db.json, __pycache__ et secrets.properties.

- Données biométriques : faces_db.json stocké localement sur le serveur de développement, exclu du dépôt pour des raisons de confidentialité.

Conclusion

La mise en œuvre de GeoQuiz Explorer a permis de concrétiser l'ensemble des fonctionnalités définies lors de la phase de conception. L'application intègre avec succès l'authentification biométrique faciale, le quiz géolocalisé, l'assistant IA conversationnel avec saisie vocale, et un backend personnel FastAPI.

Conclusion Générale et Perspectives

Ce projet a permis de concevoir et de développer GeoQuiz Explorer, une application Android native en Java intégrant des fonctionnalités avancées : authentification biométrique faciale via un backend FastAPI personnel, géolocalisation GPS, photographies interactives de rue via l'API Mapillary, backend cloud Firebase, assistant conversationnel IA avec saisie vocale, et interface Material Design 3. Le résultat est une application fonctionnelle, robuste et professionnelle qui répond pleinement aux objectifs fixés.

Sur le plan technique, ce projet a été l'occasion d'approfondir plusieurs compétences fondamentales : le cycle de vie des activités Android, l'intégration Firebase, la consommation d'APIs REST, l'utilisation de CameraX, la création d'un backend Python FastAPI, et l'intégration de modèles d'IA (DeepFace, LLaMA 3.3 70B).

Perspectives et Améliorations Futures

- Extension à d'autres villes marocaines (Rabat, Tanger, Marrakech) : l'architecture Firestore est déjà conçue pour accueillir de nouvelles collections de questions.
- Déploiement cloud du backend FastAPI : migration vers AWS EC2, Render ou Railway avec domaine statique et certificat SSL permanent.
- Sécurisation JWT : token JWT minté par FastAPI après vérification faciale, utilisé pour autoriser les accès Firestore côté serveur.
- Assistant IA basé sur RAG : indexation de documents PDF sur l'histoire des rues dans ChromaDB/FAISS.
- Mode multijoueur : défis entre amis via Firebase Realtime Database.
- Migration Kotlin + Jetpack Compose : modernisation de l'architecture.

Bibliographie et Références

Documentation Officielle

- Android Developers Documentation – developer.android.com/docs
- Firebase Documentation – firebase.google.com/docs
- CameraX – developer.android.com/training/camerax
- Material Design 3 – m3.material.io
- Mapillary API Documentation – www.mapillary.com/developer/api-documentation
- Groq API Documentation – console.groq.com/docs
- FastAPI Documentation – fastapi.tiangolo.com
- DeepFace (GitHub) – github.com/serengil/deepface
- Cloudflare Tunnel – developers.cloudflare.com/cloudflare-one/connections/connect-apps

Ouvrages de Référence

- GRIFFITHS, Dawn & GRIFFITHS, David. Head First Android Development. O'Reilly Media, 2021.
- MEIER, Reto. Professional Android 4 Application Development. Wrox Press, 2012.

Ressources en Ligne

- Stack Overflow – stackoverflow.com
- GitHub – github.com/Aymen21-n/QuizApp-GeoQuiz-Explorer-Nafis
- GitHub Copilot – github.com/features/copilot
- Medium – articles sur l'architecture Android, Firebase best practices, intégration d'APIs REST